# C# Programming for Indian Students — Chapter 2: Setting Up C#

Oct 31, 2026 · @Vikash

## 2.1 What you will learn

In Chapter 1 you read C# programs. In this chapter you set up your own computer so that you can type, build and run them. By the end you will be able to:

- name the tools needed to write C# programs and say what each one does;
- install Visual Studio Community, or the .NET SDK with Visual Studio Code;
- create a console project, build it and run it;
- use the `dotnet` command line to do the same thing;
- read the files and folders that a new project contains;
- fix the most common setup problems.

**What you need.** A laptop or desktop with an internet connection for the download, about 10 GB of free disk space for Visual Studio (the .NET SDK alone is much smaller), and an administrator login to install software. If you use a college lab, ask your lab in-charge which of these tools are already installed.

**Remember:** you will use the programs from Chapter 1 as your first test. If Program 1.1 prints `Hello, India!` on your screen, your setup is complete.

## 2.2 The tools you need

To run C# you need three things: an **SDK** that can build and run programs, an **editor** to type the code, and a **terminal** if you want to use commands. Some tools bundle several of these together.

| Tool | What it is | Gives you |
|---|---|---|
| .NET SDK | Software Development Kit: the compiler, the runtime and the `dotnet` command | Build and run C# programs |
| Visual Studio | A full **IDE** (Integrated Development Environment) from Microsoft | Editor, compiler, debugger and project templates in one window |

Vikash Kumar

| Tool | What it is | Gives you |
|---|---|---|
| Visual Studio Code | A light, free code editor that works on Windows, Linux and macOS | Editor; add the C# extension and the .NET SDK to build programs |
| Terminal / Command Prompt | A window where you type commands | Run `dotnet` commands |

**Which should you choose?**

| Your situation | Best choice |
|---|---|
| Windows laptop, want everything in one place | Visual Studio Community (this book's main tool) |
| Linux or macOS, or a low-spec laptop | .NET SDK + Visual Studio Code |
| Only want to test a tiny program quickly | .NET SDK + any text editor + terminal |

**Is it free?** Microsoft's page for Visual Studio Community describes it as a free IDE for students, open-source developers and individual developers. An unlimited number of users in an organization can use it in a classroom learning environment or for academic research. Larger companies have limits, so read the licence terms if you ever use it at work.

**IDE or editor?** An IDE such as Visual Studio does much more than an editor: it creates projects for you, shows errors as you type, and lets you pause a running program to inspect it (this is called **debugging**). Beginners usually find an IDE easier.

## 2.3 Installing Visual Studio

**Option A: Visual Studio Community (Windows)**

1. Open **visualstudio.microsoft.com** in your browser and download **Visual Studio Community** (use the latest version offered; this book was prepared with Visual Studio 2026, and Visual Studio 2022 also works if it has the .NET 10 SDK).
2. Run the downloaded installer. It is a small program called the Visual Studio Installer, which then downloads the rest.
3. When it asks which **workloads** to install, tick **.NET desktop development**. A workload is a bundle of tools for one kind of work. Microsoft's tutorial states that the .NET SDK is installed automatically when you select this workload.
4. Click **Install** and wait. The download is large, so a fast connection helps.

Vikash Kumar

5. When it finishes, start Visual Studio and sign in with a Microsoft account if asked. You can also choose to skip signing in.

**Remember:** if you later cannot see the console project templates, you probably missed the workload. Microsoft's tutorial says to choose **Install more tools and features** under the message *Not finding what you're looking for?* and add the workload then.

**Option B: .NET SDK and Visual Studio Code (Windows, Linux, macOS)**

1. Download and install the **.NET SDK** from **dotnet.microsoft.com/download**. Choose the long-term support (LTS) version, which is .NET 10 at the time of writing.
2. Download and install **Visual Studio Code** from **code.visualstudio.com**.
3. In Visual Studio Code open the Extensions panel and install the **C# Dev Kit** extension.
4. Open a terminal and check the installation with the command below.

```
dotnet --version
```

The terminal prints the SDK version number. You should see a number that starts with `10.` if you installed .NET 10, for example `10.0.100` (your last digits may differ). If you see `'dotnet' is not recognized`, restart the terminal or your computer, and install the SDK again if it still fails.

**LTS and STS.** Long-Term Support releases of .NET get free fixes for about three years and are the safe choice for college work. Standard-Term Support releases get a shorter period of support.

## 2.4 Your first project in Visual Studio

**Creating the project** (these steps follow Microsoft's console-app tutorial)

1. Start Visual Studio and, on the start page, choose **Create a new project**.
2. In the search box type **console**. Choose **C#** in the language list and **All platforms** in the platform list.
3. Select the **Console App** template and click **Next**. (Take care to pick the one for C#, not Visual Basic, and not the older .NET Framework version.)
4. In **Configure your new project**, type the project name **HelloApp** and click **Next**.
5. In **Additional information**, select **.NET 10.0 (Long Term Support)** and click **Create**.

Visual Studio creates the project and opens a file called `Program.cs`. The template has already written a complete program for you:

**Program 2.1: The code generated by the template**

Vikash Kumar

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

The first line starting with `//` is a **comment**. The compiler ignores comments; they are notes for people. The second line is the top-level-statements form that you met in Chapter 1.

### Building and running

| Action | Menu | Keyboard | What it does |
|---|---|---|---|
| Build | Build > Build Solution | `Ctrl + Shift + B` | Compiles the code and shows errors, but does not run it |
| Run without debugging | Run > Run without Debugging | `Ctrl + F5` | Runs the program and keeps the console open until you press a key |
| Start debugging | Run > Start Debugging | `F5` | Runs with the debugger attached, so breakpoints work |

Press `Ctrl + F5`. A console window opens and shows the output below.

### Output

```
Hello, World!
```

**Make it yours.** Change the text inside the quotes to `Hello, India!`, press `Ctrl + F5` again, and Visual Studio rebuilds the program automatically before running it.

**Remember:** use `Ctrl + F5` for ordinary programs. With `F5` the console window may close the moment the program finishes, so you cannot read the output. Debugging with `F5` is covered in a later chapter.

### The Visual Studio window

| Part | Where | What it shows |
|---|---|---|
| Code editor | Centre | The file you are typing in |
| Solution Explorer | Right side | All files of your project; double-click one to open it |
| Error List | Bottom | Compiler errors and warnings; double-click an error to jump to it |
| Output window | Bottom | Messages from the build |

Vikash Kumar

If a window is missing, open it from the **View** menu.

## 2.5 Using the command line

Everything Visual Studio does can also be done with the `dotnet` command. Knowing it helps on Linux and macOS, in Visual Studio Code, and in many interviews. Open a terminal (Command Prompt, PowerShell or the Terminal in VS Code) in a folder where you keep your programs.

| Command | What it does |
|---|---|
| `dotnet --version` | Shows the SDK version |
| `dotnet new console -n HelloApp` | Creates a new console project in a folder named HelloApp |
| `cd HelloApp` | Moves into that folder |
| `dotnet build` | Compiles the project |
| `dotnet run` | Builds if needed, then runs the program |

**Session**

```
> dotnet new console -n HelloApp
The template "Console App" was created successfully.

> cd HelloApp
> dotnet run
Hello, World!
```

The first command printed more lines about restoring packages when it ran on a connected computer, so your screen may show a few extra lines. The last line is the program's output. We tested the commands above with the .NET 8 SDK. The same commands work in .NET 10, but the template in .NET 10 creates the same two lines of code with a different target version number in the `.csproj` file.

To run the program you edited in Visual Studio from the command line, open the terminal in the project folder (the folder that contains the `.csproj` file) and type `dotnet run`.

**Remember:** `dotnet run` must be typed inside the project folder. If you type it in the wrong folder you get an error saying that no project file was found.

Vikash Kumar

## 2.6 Understanding the project

A new project contains only two files that you edit or care about. Building the project adds more folders.

```
HelloApp/
  HelloApp.csproj     <- project settings (you may edit)
  Program.cs          <- your code (you edit this)
  bin/                <- created after build: the finished program
    Debug/net8.0/
      HelloApp.dll    <- your compiled program (IL)
  obj/                <- created after build: temporary build files
```

The folder name under `bin/Debug` follows the version you chose, so with .NET 10 it is `net10.0`. On Windows, the same folder also holds `HelloApp.exe`, a small launcher you can double-click.

| Item | What it is | Edit it? |
|---|---|---|
| `Program.cs` | Source code, where your program starts | Yes |
| `HelloApp.csproj` | The project file, written in XML | Rarely, as a beginner |
| `bin` | Output folder with the compiled `.dll` | No |
| `obj` | Temporary files used while building | No |

### The project file

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

| Setting | Meaning |
|---|---|
| `OutputType` = `Exe` | The project builds a program that can run, not a library |

Vikash Kumar

| Setting | Meaning |
|---|---|
| `TargetFramework` | The .NET version used. With .NET 10 it reads `net10.0` |
| `ImplicitUsings` = `enable` | Common `using` lines such as `using System;` are added for you, which is why `Console` works without writing `using System;` |
| `Nullable` = `enable` | The compiler warns about values that may be missing (null). You will study this in a later chapter |

**Remember:** the `bin` and `obj` folders are created by the build. If your project behaves strangely, you can delete these two folders and build again; they will be recreated.

## 2.7 Setup problems and the mini project

### Common setup problems

| Problem | Likely cause | Fix |
|---|---|---|
| No "Console App" template in the list | The .NET workload was not installed | Re-open the Visual Studio Installer, choose **Modify**, and add **.NET desktop development** |
| `'dotnet' is not recognized` in the terminal | SDK missing, or the terminal was open during install | Install the .NET SDK, then close and reopen the terminal |
| Console window closes immediately | You pressed `F5` | Use `Ctrl + F5` |
| Error: no project file found | `dotnet run` typed in the wrong folder | Use `cd` to go into the folder that holds the `.csproj` file |
| Build fails on a project made by a friend | It targets a .NET version you have not installed | Install that SDK, or change `TargetFramework` to your version |
| Text is garbled when pasted from a PDF or website | Fancy quote marks instead of plain `"` | Retype the quotes in the editor |

**Comments.** A comment is a note that the compiler ignores. `//` starts a comment that lasts until the end of the line.

Vikash Kumar

```
// This whole line is a comment
Console.WriteLine("Hello"); // a comment can also follow code
```

### Mini project: My Introduction

**Problem.** Create a new console project named `MyIntro` and print a neat introduction card about yourself. This checks that your setup works and gives you practice with the escape sequences from Chapter 1.

### Program 2.2: My Introduction

```
// My Introduction - Chapter 2 mini project
Console.WriteLine("**********************************");
Console.WriteLine("        MY INTRODUCTION");
Console.WriteLine("**********************************");
Console.WriteLine("Name\t\t: Priya Nair");
Console.WriteLine("College\t\t: Government Engineering College");
Console.WriteLine("Course\t\t: B.Tech (Computer Science)");
Console.WriteLine("Hobby\t\t: Playing cricket");
Console.WriteLine("Language\t: C#");
Console.WriteLine("**********************************");
Console.WriteLine("Setup complete. Let's start coding!");
```

### Output

```
**********************************
        MY INTRODUCTION
**********************************
Name            : Priya Nair
College         : Government Engineering College
Course          : B.Tech (Computer Science)
Hobby           : Playing cricket
Language        : C#
**********************************
Setup complete. Let's start coding!
```

**How it works.** Two tabs ( `\t\t` ) push the text after short words such as `Name` to the same column as the longer word `Language` , which needs one tab. If your colons do not line up on your screen, the tab width differs; adjust the number of tabs.

### Steps

1. Create the project `MyIntro` in Visual Studio (or with `dotnet new console -n MyIntro` ).
2. Replace the code in `Program.cs` with Program 2.2 and change the details to your own.
3. Run it with `Ctrl + F5` or `dotnet run` .

Vikash Kumar

4. Show the output to a friend and explain each line.

## 2.8 Practice questions

Try each question on your own first. The answers follow the list.

**Section A: Short answer**

1. What is the .NET SDK, and why do you need it?
2. What is the difference between an IDE and a code editor? Give one example of each.
3. Which Visual Studio workload do you install to create console applications?
4. What is the difference between `Ctrl + F5` and `F5`?
5. What is the role of the `.csproj` file?
6. Which folder holds the compiled `.dll` file of your project?

**Section B: Commands**

7. Write the command to create a console project named `Library`.
8. Write, in order, the commands to create a project named `Marks`, move into its folder and run it.

**Section C: Predict the output**

9. What does this program print?

```
// Program Q
Console.WriteLine("Hello"); // greet
Console.WriteLine("Done");
```

**Section D: Do it yourself**

10. Create a project named `CityCard` that prints your name, city and favourite food in three lines. Run it once from Visual Studio and once with `dotnet run`.

### Answers

1. The .NET SDK (Software Development Kit) contains the compiler, the runtime and the `dotnet` command. Without it your computer cannot build or run C# programs.
2. An IDE (Visual Studio) bundles an editor, compiler, debugger and project templates. A code editor (Visual Studio Code) mainly edits text and needs extensions and the SDK to build programs.
3. **.NET desktop development**.
4. `Ctrl + F5` runs without debugging and keeps the console open until you press a key. `F5` starts debugging, and the console window may close as soon as the program ends.

Vikash Kumar

5. It is the project file. It tells the compiler what kind of program to build ( `OutputType` ), which .NET version to use ( `TargetFramework` ) and which features to switch on.
6. `bin` , inside `Debug` and then a folder named after the .NET version, for example `bin/Debug/net10.0` .
7. `dotnet new console -n Library`
8. In this order:

```
dotnet new console -n Marks
cd Marks
dotnet run
```

9. The comments are ignored, so the output is:

```
Hello
Done
```

10. One possible `Program.cs` :

```
Console.WriteLine("Name : Aarav Sharma");
Console.WriteLine("City : Pune");
Console.WriteLine("Food : Poha");
```

```
Name : Aarav Sharma
City : Pune
Food : Poha
```

## 2.9 MCQs and viva questions

**Multiple-choice questions** (answers are listed after the questions)

1. SDK stands for:
   - (a) System Debug Kit
   - (b) Software Development Kit
   - (c) Source Data Key
   - (d) Standard Design Kit
2. Which of these is an IDE?
   - (a) Notepad
   - (b) Visual Studio
   - (c) Command Prompt
   - (d) Paint

Vikash Kumar

3. The shortcut to run a program without debugging in Visual Studio is:
   - (a) `Ctrl + S`
   - (b) `F5`
   - (c) `Ctrl + F5`
   - (d) `Alt + F4`
4. Which command creates a new console project?
   - (a) `dotnet make console`
   - (b) `dotnet new console`
   - (c) `dotnet create app`
   - (d) `dotnet start`
5. Which command builds and runs the project in the current folder?
   - (a) `dotnet run`
   - (b) `dotnet open`
   - (c) `dotnet go`
   - (d) `dotnet play`
6. The project file of a C# project ends with:
   - (a) `.cs`
   - (b) `.dll`
   - (c) `.csproj`
   - (d) `.txt`
7. Which folder contains the compiled program?
   - (a) `obj`
   - (b) `bin`
   - (c) `src`
   - (d) `lib`
8. A single-line comment in C# starts with:
   - (a) `#`
   - (b) `//`
   - (c) `--`
   - (d) `**`
9. The Visual Studio window that lists all files of the project is:
   - (a) Error List
   - (b) Output window
   - (c) Solution Explorer

Vikash Kumar

- (d) Toolbox

10. A bundle of tools for one kind of development, chosen in the Visual Studio Installer, is called a:
    - (a) workload
    - (b) workbook
    - (c) workspace
    - (d) workflow

**MCQ answers:** 1-b, 2-b, 3-c, 4-b, 5-a, 6-c, 7-b, 8-b, 9-c, 10-a.

**Viva questions** (short oral questions with model answers)

1. *What is the difference between building and running?* Building compiles your code into a `.dll`. Running starts that compiled program.
2. *What does `dotnet run` do?* It builds the project if needed and then runs it.
3. *Why did `Console.WriteLine` work without `using System;` in the template?* Because `ImplicitUsings` is enabled in the project file, which adds common `using` lines automatically.
4. *What is a workload?* A bundle of tools in the Visual Studio Installer for one kind of work, such as .NET desktop development.
5. *What is LTS?* Long-Term Support: a .NET release that receives free fixes for a longer period, so it is the safer choice for study and projects.
6. *Can you write C# without Visual Studio?* Yes. The .NET SDK and any text editor or Visual Studio Code are enough.
7. *What is a comment, and does it affect the program?* It is a note for people, starting with `//`. The compiler ignores it, so it does not change the output.

## 2.10 Summary and what comes next

- You need the **.NET SDK** and an editor. **Visual Studio** is an IDE that includes everything; **Visual Studio Code** is a lighter editor.
- In Visual Studio, install the **.NET desktop development** workload, then use **Create a new project > Console App > C#**.
- Use `Ctrl + F5` to run without debugging.
- The same work can be done with `dotnet new console -n Name`, `dotnet build` and `dotnet run`.
- A project has `Program.cs` (your code) and a `.csproj` file (settings). The build creates `bin` and `obj`.
- `//` starts a comment, which the compiler ignores.

Vikash Kumar

**Next chapter:** Chapter 3, **C# Fundamentals**, introduces variables and data types: how a program stores a name, an age or a price in memory, with constants, comments and your first calculations.

Vikash Kumar